



INSTITUTO FEDERAL DA PARAÍBA
CAMPUS CAMPINA GRANDE
BACHARELADO EM ENGENHARIA DA COMPUTAÇÃO
DISCIPLINA DE LABORATÓRIO DE ESTRUTURAS DE DADOS
PROF. VICTOR ANDRÉ PINHO DE OLIVEIRA

Lab. Estruturas de Dados

Aula 10 - ED - Matrizes Esparsas

Introdução

Olá,

Na semana anterior, implementamos nossa primeira Estrutura de Dados não-Linear: a Árvore Binária. Sendo mais específico, trabalhamos a implementação de uma Árvore Binária Ordenada, também chamada de Árvore Binária de Busca.

Nesta semana, nós iremos implementar uma Estrutura de Dados, também não-Linear, chamada de **Matriz Esparsa**. É uma Estrutura voltada para armazenar grandes quantidades de dados onde boa parte deles é algum valor padrão.

Hoje não teremos recursividade, prometo!

Matrizes Esparsas

As **Matrizes Esparsas**, são Estruturas que também fazem parte do grupo de Estruturas não-Lineares.

Em geral, as **Matrizes Esparsas** são uma representação de grandes Matrizes Bidimensionais onde boa parte de seus elementos são valores padrão (p.e., 0 ou "" ou NULL). Em razão disso, uma Matriz Esparsa aloca e armazena em sua Estrutura apenas os valores diferente do padrão, proporcionando economia de memória e, eventualmente, processamento.

Implementação

Existem diversas formas de implementar Matrizes Esparsas. Algumas formas usam como base Estruturas que já estudamos na disciplina. Para esta aula, foi escolhida uma implementação que emprega Listas Simplesmente Encadeadas Ordenadas.

Em suma, teremos um *array* onde cada elemento vai representar uma Linha da Matriz. Em outras palavras, cada elemento do *array* vai apontar para a cabeça (início) de uma Lista Simplesmente Encadeada Ordenada. Essa Lista vai armazenar apenas os elementos das Colunas cujos valores sejam diferentes do valor padrão. Cada nó da

Lista, portanto, vai armazenar o número da coluna e o dado, além, claro, do ponteiro para o próximo nó.

Código

O código abaixo implementa uma Matriz Esparsa de inteiros cujo valor padrão é 0.

```
#include <stdio.h>
#include <stdlib.h>

struct sNODE{
    unsigned col;
    unsigned dado;
    struct sNODE *prox;
};

struct sMATRIZ{
    int lin, col;
    struct sNODE **m;
};

struct sMATRIZ m;

void inicializar(unsigned t_lin, unsigned t_col);

void atribuir(unsigned lin, unsigned col, int dado);
int acessar(unsigned lin, unsigned col);

int tamanho();
void imprimir();
void apagar();

int main(void) {
    return 0;
}

void inicializar(unsigned t_lin, unsigned t_col){
    m.lin = t_lin;
    m.col = t_col;
    m.m = (struct sNODE **) malloc(sizeof(struct sNODE *) *
    m.lin);

    for (int i = 0 ; i < m.lin ; i++)
```

```

        m.m[i] = NULL;
    }

void atribuir(unsigned lin, unsigned col, int dado) {
    if (lin < m.lin && col < m.col) {
        struct sNODE *aux = m.m[lin], *ant = NULL;

        while (aux && col > aux->col) {
            ant = aux;
            aux = aux->prox;
        }

        if (aux && aux->col == col){
            if (dado == 0) {
                if (ant == NULL) m.m[lin] = aux->prox;
                else ant->prox = aux->prox;

                free(aux);
            } else aux->dado = dado;

            } else if (dado != 0) {
                struct sNODE *novo = (struct sNODE *)
malloc(sizeof(struct sNODE));
                novo->col = col;
                novo->dado = dado;
                novo->prox = aux;
                if (ant == NULL) m.m[lin] = novo;
                else ant->prox = novo;
            }

        } else {
            printf("Não é possível inserir elementos fora da
Matriz.\n");
            exit(0);
        }
    }

int acessar(unsigned lin, unsigned col) {
    if (lin < m.lin && col < m.col) {
        struct sNODE *aux = m.m[lin];
    }
}

```

```

    while (aux && col > aux->col)
        aux = aux->prox;

    if (aux && col == aux->col)
        return aux->dado;

    return 0;
} else {
    printf("Não é possível acessar elementos fora da
Matriz.\n");
    exit(0);
}
}

int tamanho() {
    return m.lin * m.col;
}

void imprimir(){
    printf("M[\n");
    for (int i = 0 ; i < m.lin ; i++) {
        struct sNODE *aux = m.m[i];

        for (int j = 0 ; j < m.col ; j++) {
            if (aux && j == aux->col) {
                printf("%4d ",aux->dado);
                aux = aux->prox;
            } else printf("%4d ",0);
        }
        printf("\n");
    }
    printf("]\n");
}

void apagar(){
    for (int i = 0 ; i < m.lin ; i++) {
        struct sNODE *aux = m.m[i], *ant = NULL;

        while (aux) {
            ant = aux;

```

```

        aux = aux->prox;
        free(ant);
    }
}
free(m.m);
}

```

Para implementar essa versão de Matriz Esparsa, vamos precisar de:

- um registro chamado sNODE para armazenar as informações de um nó:
 - campo **dado**: vai armazenar um valor da Matriz Esparsa
 - campo **col**: vai armazenar o número da Coluna referente ao dado da Matriz Esparsa
 - campo **prox**: vai armazenar o endereço do próximo nó.
- um registro chamado sMATRIZ para armazenar as informações referentes à Matriz Esparsa:
 - campo **lin**: vai armazenar o total de Linhas na Matriz
 - campo **col**: vai armazenar o total de Colunas na Matriz
 - campo **m**: vai armazenar um *array* de ponteiros para sNODE. Note que m é um ponteiro de ponteiro, porque cada elemento do *array* irá apontar para a cabeça de uma Lista Encadeada. O campo m deverá ser alocado antes da utilização da Matriz.
- uma função de inicialização:
 - **inicializar**: para inicializar a Matriz, principalmente, alocando espaço para o campo m de sMATRIZ.
- duas funções base:
 - **atribuir**: função usada para modificar um valor de um determinado elemento (ou célula) da Matriz. Note que, por se tratar de uma Matriz Esparsa, onde a regra para o valor dos elementos é o valor padrão, não se insere um valor, mas se atribui um valor à uma célula da Matriz, alterando o valor padrão ou o antigo valor
 - **acessar**: para obter um elemento específico da Matriz.
- algumas funções auxiliares:
 - **tamanho**: para retornar o tamanho da Matriz
 - **imprimir**: para imprimir a Matriz
 - **apagar**: para apagar toda a memória que foi alocada para a Estrutura.

Por razões didáticas, nosso código trabalha apenas com uma Matriz Esparsa, a matriz m, declarada de forma global.

Vamos entender como cada função funciona.

Função de inicialização

Eis o código da função de inicialização.

```

void inicializar(unsigned t_lin, unsigned t_col){
    m.lin = t_lin;
    m.col = t_col;
    m.m = (struct sNODE **) malloc(sizeof(struct sNODE *) *
m.lin);

    for (int i = 0 ; i < m.lin ; i++)
        m.m[i] = NULL;
}

```

A função não retorna nada e recebe o tamanho da Matriz em linhas (t_lin) e colunas (t_col). Primeiro atribuímos t_lin ao campo lin e t_col ao campo col da Matriz Esparsa m. Em seguida, alocamos o *array* de ponteiros em função da quantidade de linhas na Matriz.

Depois da alocação, percorremos o *array* para setar todos os elementos como NULL. Mais uma vez, perceba que o campo m é um ponteiro de ponteiro para sNODE. Em outras palavras, para nossos propósitos, você pode enxergá-lo como uma *array* de ponteiros para sNODE. Na verdade, o significado é ainda mais amplo: um *array* de ponteiros para Listas Encadeadas :).

Funções base

Vejamos agora as funções base da Estrutura.

Função atribuir

A função atribuir tem o objetivo de modificar o valor de uma célula na Matriz.

```

void atribuir(unsigned lin, unsigned col, int dado) {
    if (lin < m.lin && col < m.col) {
        struct sNODE *aux = m.m[lin], *ant = NULL;

        while (aux && col > aux->col) {
            ant = aux;
            aux = aux->prox;
        }

        if (aux && aux->col == col){
            if (dado == 0) {
                if (ant == NULL) m.m[lin] = aux->prox;
                else ant->prox = aux->prox;

                free(aux);
            }
        }
    }
}

```

```

        } else aux->dado = dado;

    } else if (dado != 0) {
        struct sNODE *novo = (struct sNODE *)
malloc(sizeof(struct sNODE));
        novo->col = col;
        novo->dado = dado;
        novo->prox = aux;
        if (ant == NULL) m.m[lin] = novo;
        else ant->prox = novo;
    }

} else {
    printf("Não é possível inserir elementos fora da
Matriz.\n");
    exit(0);
}
}

```

A função atribuir não retorna nada e recebe a posição (lin e col) e o dado a ser atribuído.

Primeiramente, verificamos se a posição diz respeito a uma posição válida na Matriz, isto é, se está dentro dos limites. Em caso negativo, avisamos ao usuário e encerramos o programa. Em caso positivo, seguimos para atribuir o dado na posição indicada.

Considere que para acessar uma linha da Matriz é necessário indexar o array m de sMATRIZ com a linha (lin). E para acessar um elemento na coluna é preciso percorrer a Lista Encadeada buscando pela coluna indicada (col). Atente que se a linha contiver apenas valores padrão, então a Lista estará vazia. Se apenas um elemento da linha for diferente do padrão, então a Lista conterá apenas um nó (que armazena a coluna e o dado)... e por aí vai.

A lógica para encontrar a posição para atribuir o valor na Matriz é bem semelhante à lógica de quando Estudamos Listas Simplesmente Encadeadas Ordenadas. Porém, ao invés de considerar o dado, consideramos a coluna.

Primeiramente, armazenamos em aux o início da Lista indexando o array m de sMATRIZ com a linha (lin) e também declaramos e setamos um ponteiro ant para NULL. Em seguida, fazemos o ponteiro aux correr pela lista enquanto a coluna (col) for maior que a coluna (aux->col) do nó que estamos visitando.

Depois que o while termina, verificamos se o nó corresponde à coluna (col) que estamos querendo atribuir o dado. Se sim, então já existe algum valor que é diferente do padrão, i.e., diferente de zero. Assim, verificamos se o dado que estamos querendo inserir é zero, pois, em caso afirmativo, devemos apagar o nó e fazer os devidos ajustes

nos ponteiros da Lista. Caso contrário, se o dado que estamos querendo inserir é diferente de zero, então simplesmente atribuímos ao campo dado do nó. É preciso verificar se `ant == NULL` para ajustar o ponteiro para a cabeça da Lista. Caso `ant` não seja `NULL`, então fazemos com que `ant->prox` aponte para `aux->prox`, pois o nó `aux` será desalocado.

Continuando, se não achamos a coluna exata, então o `while` parou no ponto em que vamos inserir um novo nó. Porém, só é preciso inserir esse novo nó se o dado que estamos querendo inserir for diferente de zero. Nesse caso, alocamos um novo nó e setamos seus campos. O novo nó terá `aux` como próximo nó. Verificamos se `ant == NULL` para atribuir o novo nó à cabeça da Lista. Se `ant` não for `NULL`, então fazemos `ant->prox` apontar para o novo nó. Ou seja, o novo nó ficará entre `ant` e `aux`.

Função acessar

A função `acessar` tem o objetivo de retornar o valor de uma posição da Matriz.

```
int acessar(unsigned lin, unsigned col) {
    if (lin < m.lin && col < m.col) {
        struct sNODE *aux = m.m[lin];

        while (aux && col > aux->col)
            aux = aux->prox;

        if (aux && col == aux->col)
            return aux->dado;

        return 0;
    } else {
        printf("Não é possível acessar elementos fora da
Matriz.\n");
        exit(0);
    }
}
```

A função `acessar` retorna o valor em função da posição (`lin` e `col`) passada como argumento. Em linhas gerais, a lógica da função `acessar` é a seguinte: buscamos a posição passada na Estrutura e retornamos o valor caso o encontremos. Caso contrário, retornamos o valor padrão, que é zero.

Primeiramente, verificamos se a posição passada corresponde à uma posição válida na Matriz, igual como foi feito na função `atribuir`.

Se a posição for válida, indexamos o campo `m` da `sMATRIZ` com a linha passada (`lin`) para obter o início (cabeça) da Lista Encadeada e armazenar em `aux`. Em seguida, fazemos o ponteiro `aux` percorrer a Lista enquanto a coluna passada (`col`) for maior que a coluna do nó visitado (`aux->col`). Depois que o `while` termina, verificamos se paramos

em um nó cuja coluna corresponde à que estamos procurando (col). Se sim, retornamos o valor correspondente. Senão, retornamos o valor padrão.

Funções de auxiliares

E agora, veremos as funções auxiliares.

Função tamanho

Retorna o tamanho da Matriz.

```
int tamanho() {  
    return m.lin * m.col;  
}
```

Essa é a função mais simples da Estrutura. Simplesmente retorna o produto entre o número de linhas e colunas como tamanho da Matriz. Note que o tamanho da Matriz sempre será esse produto, mesmo que não tenhamos atribuído nenhum valor à Estrutura.

Função imprimir

Apresenta na tela a Matriz.

```
void imprimir(){  
    printf("M[\n");  
    for (int i = 0 ; i < m.lin ; i++) {  
        struct SNODE *aux = m.m[i];  
  
        for (int j = 0 ; j < m.col ; j++) {  
            if (aux && j == aux->col) {  
                printf("%4d ",aux->dado);  
                aux = aux->prox;  
            } else printf("%4d ",0);  
        }  
        printf("\n");  
    }  
    printf("]\n");  
}
```

A função imprimir tem o objetivo de mostrar na tela os valores da Matriz. A ideia é que, para cada posição, se houver algum valor atribuído, então ele será mostrado. Senão o valor padrão será exibido.

Usamos dois for aninhados. O mais externo varre as linhas, atribuindo a cabeça da Lista para aux. E o mais interno varre as Colunas. Note a forma como varremos as Colunas. Usamos o j para controlar o índice das Colunas, e em cada loop verificamos se j é igual ao índice do nó visitado. Se sim, ele é apresentado na tela e aux avança para o próximo

nó. Caso contrário, o valor padrão é apresentado na tela. Envolvemos a Matriz por “M[...]

Função apagar

Apaga toda a memória alocada pela Estrutura.

```
void apagar(){
    for (int i = 0 ; i < m.lin ; i++) {
        struct sNODE *aux = m.m[i], *ant = NULL;

        while (aux) {
            ant = aux;
            aux = aux->prox;
            free(ant);
        }
        free(m.m);
    }
}
```

Primeiramente varremos todas as Linhas da Matriz, ou seja, varremos o *array* m para atribuir a aux a cabeça de cada Lista Encadeada. Usando a lógica empregada quando estudamos Listas Simplesmente Encadeadas, apagamos cada nó da Lista. Depois de apagar todas as Listas, apagamos o *array* m que foi alocado na função inicializar.

Na aula de hoje vimos mais uma Estrutura não-Linear: as Matrizes Esparsas.

A partir da próxima aula veremos algoritmos de Ordenação.



Bons estudos e até a próxima!